
Numerical Methods for Differential Equations



INDIAN INSTITUTE OF SCIENCE EDUCATION AND RESEARCH KOLKATA

Project Report

Kalaap Sadhu

Under the guidance of Dr. Rajib Dutta, IISER Kolkata

Part of a series of notes available at kalaapks.github.io/notes.html

Last updated: September 21, 2026

List of Figures

Contents

1	ROOT FINDING ALGORITHMS	1
1.1	The concept or errors	1
1.2	Solution of linear equation	1
1.2.1	Gauss-Elimination	1
1.2.2	Gauss-Seidel and Jacobi methods	4
1.3	Solution of non-linear equation	7
1.3.1	Bisection method	7
1.3.2	Regula-Falsi method	9
1.3.3	Newton's method	11
2	DATA FITTING TECHNIQUES	13
2.1	Section 1	13
2.1.1	Subsection 1	13
3	NUMERICAL CALCULUS TECHNIQUES	14
3.1	Section 1	14
3.1.1	Subsection 1	14
4	NUMERICAL DIFFERENTIAL EQUATIONS	15

ROOT FINDING ALGORITHMS

1

1.1 The concept or errors

1.2 Solution of linear equation

1.2.1 Gauss-Elimination

We solve equations of the form $f(x_1, \dots, x_n) = 0$ where this forms a linear system:

$$\begin{aligned} a_{11}x_1 + \dots + a_{1n}x_n - b_1 &= 0 \\ &\vdots \\ a_{m1}x_1 + \dots + a_{mn}x_n - b_m &= 0 \end{aligned}$$

Further, we write $A = (a^1, \dots, a^n)_{m \times n}$. Recall, $A\vec{x} = \vec{b}$ is how we generally write it. Let us review the basic properties of solution based on dimensions of the matrices.

- $\rho(A) > \rho(A|b)$ is impossible!
- $\rho(A) < \rho(A|b) \implies$ no solution since $b \notin \text{Range}(a^1, \dots, a^n)$.
- $\rho(A) = \rho(A|b) = n \implies$ unique solution.
- $\rho(A) = \rho(A|b) < n \implies$ infinitely many solutions since $b \in \text{Range}(a^1, \dots, a^n)$ and some columns collapse.

Also since $\rho(A) \leq \min(m, n) \implies m < n$ and $n < m$ cases are covered already:

- $m < n$ (less equations, more unknowns): $\rho(A) \leq m < n$, so it no longer can have a unique solution. $\rho(A|b) > \rho(A) \implies$ no solution, and $\rho(A|b) = \rho(A) < n \implies$ infinitely many solutions.
- $m > n$ (more equations, less unknowns): $\rho(A) \leq n < m$. More equations could always be made by repeating equations in a system that has no solution / infinitely many / unique; so naturally for $m > n$ we have all possibilities.

We put $\rho(A|b)$ in various places relative to $\rho(A)$, n , and beyond n :

- $\rho(A) = \rho(A|b) < n \implies$ infinitely many solutions.
- $\rho(A) < \rho(A|b) \leq n \implies$ no solution.
- $\rho(A) < n = \rho(A|b) \implies$ no solution!
- $\rho(A) < n < \rho(A|b) \implies$ no solution.
- $n = \rho(A) = \rho(A|b) \implies$ unique solution.

We solve for a solution, when we have one, by Gauss elimination. This is the standard method we see in Linear Algebra. Let us see the number of floating-point operations (#FLOPs) a computer does in general for implementing this method.

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right] \xrightarrow[\alpha_1 = \frac{a_{21}}{a_{11}}]{R'_2 = R_2 - \alpha_1 R_1} \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & a_{m2}^{(1)} & \cdots & a_{mn}^{(1)} & b_m^{(1)} \end{array} \right]$$

One Gauss step requires:

- 1 extra multiplication ($\sim$ division) to get $\alpha_1 = \frac{a_{21}}{a_{11}}$.
- 1 A & 1 M (i.e., 1 addition/subtraction and 1 multiplication/division) to get $a_{2j}^{(1)}$ from a_{2j} for each $j = 2, \dots, n$ and b_2 , that is, n operations. The first 0 is not calculated at all and the machine simply purts it since making it 0 was the goal of $R_2 - \alpha_1 R_1$ operation to begin with.
- To get the entire first column zeroed, we do this $m - 1$ times. Hence, to get here we do $(m - 1)n$ additions and $(m - 1)(n + 1)$ multiplications.

Remark 1.1. Now, $a_{22}^{(1)}$ is our pivot entry—that is, we use it to make the column below it 0. Note that the matrix we wrote above is all that is there in computer memory, and all memory of original A is gone since keeping that in memory is pointless. In the following matrix, even though we write $a_{22}^{(1)}$ to separate it from a_{22} for us humans, there honestly isn't any need for that to a machine; $a_{22}^{(1)}$ is not only our new $(2, 2)$ -th entry but also the only entry to the machine.

Next step:

$$\left[\begin{array}{ccccc|c} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} & b_1 \\ 0 & a_{22}^{(1)} & a_{23}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ 0 & 0 & a_{33}^{(2)} & \cdots & a_{3n}^{(2)} & b_3^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & a_{m3}^{(2)} & \cdots & a_{mn}^{(2)} & b_m^{(2)} \end{array} \right]$$

Here, 1 A & 1 M are done $(n - 1)$ times ($a_{32}^{(1)} \rightarrow 0$ is known). Like before, for one row the number of FLOPs is $(n - 1)$ A and $(n - 1) + 1$ M, where +1 is due to the multiplication/division done to get the α factor for this step. Since there are $m - 2$ rows in total, the number of FLOPs is $(m - 2)(n - 1)$ additions and $(m - 2)n$ multiplications.

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a_{22} & \cdots & a_{2n} & b_2 \\ 0 & 0 & \cdots & a_{3n} & b_3 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & a_{nn} & b_n \end{array} \right]$$

When reaching our target upper triangular matrix, note that if we are dealing with a rectangular matrix $n < m$ (or $m < n$), the process will stop before m (or n) steps because both $n - j$ and $m - i$ are involved and one of them will become 0. Naturally, for a rectangular matrix the number of calculations will be lower than for a square matrix. Let us assume $m = n$ to find the maximum #FLOPs.

#FLOPs for addition/subtraction (A):

$$\begin{aligned} \text{Total A} &:= (n - 1)n + (n - 2)(n - 1) + \cdots + 1 \cdot 2 \\ &= \sum_{j=1}^{n-1} j(j + 1) \sim \mathcal{O}\left(\frac{n^3}{3}\right) \end{aligned}$$

#FLOPs for multiplication/division (M):

$$\begin{aligned} \text{Total M} &:= (n - 1)(n + 1) + (n - 2)n + \cdots + 2 \cdot 4 + 1 \cdot 3 \\ &= \sum_{i=1}^{n-1} i(i + 2) \sim \mathcal{O}\left(\frac{n^3}{3}\right) \end{aligned}$$

If the number of variables increase by one, the order of effort the machine puts for finding the solution increases by $\frac{2}{3}n^3$. That is a lot of machine effort being used. Moreover, we are not even done yet! We further do the backward substitution to get the solution $\vec{x}$:

$$\begin{aligned} x_n &= \frac{b_n}{a_{nn}} \quad \longrightarrow \quad 0 \text{ A, } 1 \text{ M} \\ a_{n-1,n-1}x_{n-1} + a_{n-1,n}x_n &= b_{n-1} \implies x_{n-1} = \frac{1}{a_{n-1,n-1}} \left(b_{n-1} - a_{n-1,n} \frac{b_n}{a_{nn}} \right) \quad \longrightarrow \quad 1 \text{ A, } 2 \text{ M} \\ \text{Similarly, } x_{n-2} &= \frac{1}{a_{n-2,n-2}} (b_{n-2} - a_{n-2,n-1}x_{n-1} - a_{n-2,n}x_n) \quad \longrightarrow \quad 2 \text{ A, } 3 \text{ M} \\ &\vdots \\ \text{FLOPs} &:= \sum_{j=0}^{n-1} [j + (j + 1)] = \sum_{j=0}^{n-1} (2j + 1) \sim \mathcal{O}\left(\frac{n^2}{2}\right) \end{aligned}$$

The overall effort to solve $A\vec{x} = \vec{b}$ by Gauss-Elimination method is $\frac{n^2}{2} + \frac{2}{3}n^3$.

We understand already that we need to come up with better algorithms for practical uses. Before that, we will see a modification of Gauss-Elimination that works on special matrices called **Banded matrices**. They show up all the time in engineering (such as the heat equation, Black-Scholes, etc.), or more generally, whenever we discretize a differential equations take place and end up with a dependency on the previous and next steps. This is the **Thomas Algorithm**.

$$\left[\begin{array}{cccccc|c} a_{11} & a_{12} & 0 & 0 & \cdots & 0 & b_1 \\ a_{21} & a_{22} & a_{23} & 0 & \cdots & 0 & b_2 \\ 0 & a_{32} & a_{33} & a_{34} & \cdots & 0 & b_3 \\ 0 & 0 & a_{43} & a_{44} & \cdots & 0 & b_4 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & a_{nn} & b_n \end{array} \right] \longrightarrow \left[\begin{array}{cccccc|c} 1 & a_{12}^{(1)} & 0 & 0 & \cdots & 0 & b_1^{(1)} \\ 0 & a_{22}^{(1)} & a_{23} & 0 & \cdots & 0 & b_2^{(1)} \\ 0 & a_{32} & a_{33} & a_{34} & \cdots & 0 & b_3 \\ 0 & 0 & a_{43} & a_{44} & \cdots & 0 & b_4 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & a_{nn} & b_n \end{array} \right]$$

- We divide the first row by a_{11} . So, we get $\{1, a_{12}/a_{11} \sim a_{12}^{(1)}, 0, \dots, b_1/a_{11} \sim b_1^{(1)}\}$; hence we only perform 2 M FLOPs (because the first 1 is simply placed there).
- In step 2, we use 1 to make $a_{21} \rightarrow 0$. We simply put 0 there. To get $a_{22}^{(1)}$ from a_{22} and $b_2^{(1)}$ from b_2 , we do 1 M and 1 A each. There are no changes in a_{23} since the entry above it is 0. Total changes: 2 M and 2 A.
- In step 3, the third row will also have 1 M and 1 A each, but something changes from this row onward:

$$a_{33}^{(1)} = a_{33} - c a_{23}, \quad \text{where } c = \frac{a_{32}}{a_{22}^{(1)}}$$

This division was not needed earlier since we had 1 in a_{11} . Row 3 onwards will thus require an extra M.

$$\begin{array}{lll} \text{1st row:} & 0 \text{ A} & 2 \text{ M} \\ \text{2nd row:} & 2 \text{ A} & 2 \text{ M} \\ \text{3rd row:} & 2 \text{ A} & 2 \text{ M} + 1 \text{ M} \\ & \vdots & \\ \text{n-th row:} & 2 \text{ A} & 2 \text{ M} + 1 \text{ M} \end{array}$$

$$\text{Total additions/subtractions} = 2(n-1) = 2n-2$$

$$\text{Total multiplications/divisions} = 3(n-2) + 4 = 3n-2$$

$$\text{Total \#FLOPs} = (2n-2) + (3n-2) = 5n-4 \sim \mathcal{O}(5n) \quad (\text{linear only!})$$

This concludes the discussion on what we call the **Direct method**. From linear algebra, this is THE method learned to solve $A\vec{x} = \vec{b}$.

1.2.2 Gauss-Seidel and Jacobi methods

Now, we consider another class of methods: **Indirect methods / Iterative methods**. These are the improved algorithm for machine we were talking about earlier.

- Gauss-Seidel Method
- Jacobi Method

Both methods are very similar and geometrically motivated. We start with a simple example of a system with 2 variables and 2 unknowns:

$$\begin{array}{l} x_1 + 2x_2 = 1 \\ x_1 - x_2 = 4 \end{array}$$

Rearrangement 1: Let us start with first equation and make x_1 the subject of the 1st equation, then x_2 the subject of the 2nd:

$$\begin{aligned}x_1 &= 1 - 2x_2 \\x_2 &= x_1 - 4\end{aligned}$$

Rearrangement 2: Let us again start with the second equation now and make x_1 the subject of this equation, then x_2 will be the subject of the other:

$$\begin{aligned}x_1 &= 4 + x_2 \\x_2 &= \frac{1 - x_1}{2}\end{aligned}$$

Firstly, note that there are two more rearrangements. We pick one of the equations and pick which variable will be our subject; this choice matters to us.

Let the initial guess be $(x_1^{(0)}, x_2^{(0)})$.

Let us start with **Rearrangement 1** written as an iterative algorithm:

$$\begin{cases} x_1^{(i+1)} = 1 - 2x_2^{(i)} \\ x_2^{(i+1)} = x_1^{(i+1)} - 4 \end{cases}$$

Put $x_2^{(0)}$ to get $x_1^{(1)}$. Then put $x_1^{(1)}$ to get $x_2^{(1)}$ in the 2nd equation (hence the algorithm has a lag in the 1st equation, but none in the 2nd). We will now apply this iteratively. Put $x_2^{(1)}$ in the 1st equation again to get $x_1^{(2)}$, then put $x_1^{(2)}$ into the 2nd to get $x_2^{(2)}$, and so on:

$$(x_1^{(0)}, x_2^{(0)}) \longrightarrow (x_1^{(1)}, x_2^{(0)}) \longrightarrow (x_1^{(1)}, x_2^{(1)}) \longrightarrow (x_1^{(2)}, x_2^{(1)}) \longrightarrow (x_1^{(2)}, x_2^{(2)}) \longrightarrow \dots$$

At the end of every full cycle, we get an updated candidate for the solution:

$$\begin{aligned}(0, 0) &\implies x_1 = 1 - 0 = 1 \implies x_2 = 1 - 4 = -3 \longrightarrow (1, -3) \\(1, -3) &\implies x_1 = 1 + 6 = 7 \implies x_2 = 7 - 4 = 3 \longrightarrow (7, 3) \\(7, 3) &\implies x_1 = 1 - 6 = -5 \implies x_2 = -5 - 4 = -9 \longrightarrow (-5, -9) \\(-5, -9) &\implies x_1 = 1 + 18 = 19 \implies x_2 = 19 - 4 = 15 \longrightarrow \text{not converging!}\end{aligned}$$

Something went wrong! For now, let's try the same algorithm with **Rearrangement 2** and see if it works out:

$$\begin{cases} x_1^{(i+1)} = 4 + x_2^{(i)} \\ x_2^{(i+1)} = \frac{1 - x_1^{(i+1)}}{2} \end{cases}$$

$$\begin{aligned}(0, 0) &\implies x_1 = 4 \implies x_2 = \frac{1 - 4}{2} = -1.5 \longrightarrow (4, -1.5) \\(4, -1.5) &\implies x_1 = 4 - 1.5 = 2.5 \implies x_2 = \frac{1 - 2.5}{2} = -0.75 \longrightarrow (2.5, -0.75) \\(2.5, -0.75) &\implies x_1 = 4 - 0.75 = 3.25 \implies x_2 = \frac{1 - 3.25}{2} = -1.125 \longrightarrow (3.25, -1.125)\end{aligned}$$

This sequence actually converges to the true solution $(3, -1)$.

Keep in mind that in reality, the true solution is not known beforehand; so if the process is converging or diverging needs to be addressed properly; Also, it is unknown which rearrangement will work for us and that is determined by something called the **Diagonal Dominance condition**. The stopping condition is determined by a preset tolerance error margin ε_a (approximate relative error).

Geometrically, Gauss–Seidel operates as follows: Starting from the initial guess $(x_1^{(0)}, x_2^{(0)})$, Rearrangement 1 hits the first line (the equation with lag) and then alternates between the lines, diverging outward. Rearrangement 2 hits the line corresponding to its first update and spirals inwards to the intersection point $(3, -1)$.

There are in total four traversal possibilities depending on which line and which axis direction is updated first:

- Hit the blue line along the x_1 axis.
- Hit the blue line along the x_2 axis.
- Hit the red line along the x_1 axis.
- Hit the red line along the x_2 axis.

To standardize the algorithm, we fix the update order: $x_1 \rightarrow x_2$.

Before stating the general n -variable scheme, consider the 3-variable case:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \end{cases}$$

Rewriting:

$$\begin{aligned} x_1 &= \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3) \\ x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3) \\ x_3 &= \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2) \end{aligned}$$

With initial guess $(x_1^{(0)}, x_2^{(0)}, x_3^{(0)})$ and update sequence $x_1 \rightarrow x_2 \rightarrow x_3$:

- Put $x_2^{(0)}, x_3^{(0)}$ to get $x_1^{(1)}$.
- Use new $x_1^{(1)}$ and old $x_3^{(0)}$ to get $x_2^{(1)}$.
- Use new $x_1^{(1)}$ and new $x_2^{(1)}$ to get $x_3^{(1)}$.

Then treat $(x_1^{(1)}, x_2^{(1)}, x_3^{(1)})$ as the new initial point and repeat:

$$\begin{aligned} (0, 0, 0) &\rightarrow (x_1^{(1)}, 0, 0) \rightarrow (x_1^{(1)}, x_2^{(1)}, 0) \rightarrow (x_1^{(1)}, x_2^{(1)}, x_3^{(1)}) \\ (x_1^{(1)}, x_2^{(1)}, x_3^{(1)}) &\rightarrow (x_1^{(2)}, x_2^{(1)}, x_3^{(1)}) \rightarrow (x_1^{(2)}, x_2^{(2)}, x_3^{(1)}) \rightarrow (x_1^{(2)}, x_2^{(2)}, x_3^{(2)}) \rightarrow \dots \end{aligned}$$

The General n -variable Gauss–Seidel Scheme: For $\sum_{j=1}^n a_{ij}x_j = b_i, i = 1, \dots, n$:

$$x_i^{(m+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(m+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(m)} \right), \quad i = 1, \dots, n$$

This is well-defined since $x_1^{(m+1)}, \dots, x_{i-1}^{(m+1)}$ are already computed when calculating $x_i^{(m+1)}$. We continue until the step change satisfies a prescribed tolerance.

The Jacobi Scheme: The Jacobi scheme updates all components simultaneously using only values from the previous iteration:

$$x_i^{(m+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(m)} - \sum_{j=i+1}^n a_{ij}x_j^{(m)} \right), \quad i = 1, \dots, n$$

For $n = 3$:

$$\begin{aligned} x_1^{(1)} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(0)} - a_{13}x_3^{(0)}) \\ x_2^{(1)} &= \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(0)} - a_{23}x_3^{(0)}) \quad (\text{does not use updated } x_1) \\ x_3^{(1)} &= \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(0)} - a_{32}x_2^{(0)}) \quad (\text{does not use updated } x_1, x_2) \end{aligned}$$

While Gauss–Seidel uses updated coordinates immediately, Jacobi computes all entries from the old point before updating the full state vector.

1.3 Solution of non-linear equation

Here we consider $f(x_1, \dots, x_n) = (0, \dots, 0) \in \mathbb{R}^m$, where equations contain non-linear terms such as polynomials, $\ln(\cdot)$, $\sin(\cdot)$, $\cos(\cdot)$, etc. This introduces systems that can have multiple isolated roots:

$$f(\vec{x}) = \begin{bmatrix} f_1(\vec{x}) \\ \vdots \\ f_m(\vec{x}) \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$$

We begin with the single-variable case $f(x) = 0$.

1. **Bracketing Methods** (require two initial guesses that bracket a root):

- Bisection Method
- Regula-Falsi Method

2. **Open Methods:**

- Newton–Raphson Method
- Secant Method
- Fixed-Point Iteration

1.3.1 Bisection method

Assume f is continuous in $[a, b]$ and also $f(a)f(b) < 0$, then it will have one root in $[a, b]$. We start with two points x_1, x_2 and if we have a sign change, we have a root. Then pick the middle point $x_0 = \frac{x_1 + x_2}{2}$ and look for a sign change in $[x_1, x_0]$ and $[x_0, x_2]$.

Bisection method is effective but convergence is slow.

Define error as the difference between upper & lower bound of the root (i.e., size of interval in which we are looking for sign change). Then at every iteration, the error is halved & hence convergence rate is linear.

At k^{th} step error is $|b - a|$, then in $(k + 1)^{\text{th}}$ step it is $\frac{|b-a|}{2}$, and in $(k + 2)^{\text{th}}$ step it is $\frac{|b-a|}{4}$, i.e.,

$$\begin{aligned} 1^{\text{st}} \text{ step error} &= l \\ 2^{\text{nd}} \text{ step error} &= \frac{l}{2} \\ 3^{\text{rd}} \text{ step error} &= \frac{2^{\text{nd}} \text{ step error}}{2} \\ &\vdots \\ k^{\text{th}} \text{ step error} &= \frac{(k-1)^{\text{th}} \text{ step error}}{2} \Rightarrow \mathcal{E}_k = \frac{1}{2} \mathcal{E}_{k-1} \quad \text{which is linear.} \end{aligned}$$

Example 1.2. Let f be a polynomial,

$$f(x) = a_n x^n + \cdots + a_1 x + a_0$$

The range of the root is given by $[-x_{\max}, x_{\max}]$ and we will initiate bisection with $-x_{\max}$ and $x_{\max}$. From relation between roots & coefficients we know,

$$\sum x_i = -\frac{a_{n-1}}{a_n}, \quad \sum_{i < j} x_i x_j = \frac{a_{n-2}}{a_n}$$

Since,

$$\begin{aligned} (x_1 + \cdots + x_n)^2 &= \sum_{i \neq j} x_i x_j + \sum x_i^2 \\ \text{i.e., } \left(\sum x_i \right)^2 - 2 \sum_{i < j} x_i x_j &= \sum x_i^2 := x_{\max}^2 \text{ (let)} \\ \text{i.e., } x_{\max} &= \sqrt{\sum x_i^2} \\ \text{i.e., } x_{\max} &= \sqrt{\left(\frac{a_{n-1}}{a_n} \right)^2 - 2 \left(\frac{a_{n-2}}{a_n} \right)} \end{aligned}$$

$x_{\max}$ is our bound for roots because clearly for any root x_k , $x_k \leq \sqrt{\sum x_i^2}$.

Example 1.3. Consider the polynomial $f(x) = x^2 - 4x - 10$. Then we have,

$$x_{\max} = \sqrt{\left(-\frac{4}{1} \right)^2 - 2 \left(-\frac{10}{1} \right)} = \sqrt{36} = 6.$$

x	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6
$f(x)$	50	35	22	11	2	-5	-10	-13	-14	-13	-10	-5	2

We have at least one root in $[-2, -1]$ and in $[5, 6]$.

Let's apply our algorithm in $[-2, -1]$ only.

Midpoint of $[-2, -1]$ is -1.5 and $f(-1.5) = -1.75 < 0$.

So, sign change is happening in $[-2, -1.5]$ instead of in $[-1.5, -1]$. We can further zoom in on $[-2, -1.5]$ interval. The midpoint is -1.75 and $f(-1.75) = 0.0625 > 0$.

x	$f(x)$
-2	2
-1.75	0.0625
-1.50	-1.75
-1.25	need not calculate
-1	-5

Root lies in $[-1.75, -1.50]$. Using computer programme now we can get as close to the root as we want since length of the interval is getting halved every time.

After few iterations approximate root is -1.7416 .

Example 1.4. Solving $2 - x + \ln(x) = 0$ (Excel codes)

A	B	C	D	E	F	G	H
#	$x(l)$	$x(r)$	$f(l)$	$f(r)$	$x(i+1)$	$f(i+1)$	$f(l)f(i+1)$
1	1	4	$= (2 - B2 + \ln(B2))$				
2	$= \text{IF}(H1 > 0, F2, B2)$	$= \text{IF}(H1 > 0, C2, F2)$					

General form: $\text{IF}(\text{logic}, [\text{if true}], [\text{if false}])$.

- Column B evaluation: says if value of f at $x(l)$ & at $x(i+1)$ are not oppositely signed update $x(l)$ by $x(i+1)$.
- Column C evaluation: says if value of f at $x(l)$ & at $x(i+1)$ are not oppositely signed keep $x(r)$ or else update.

x_l, x_r are usual place holders for left & right end pts in bracketing methods.

$x(i+1)$ are simply $= (B1 + C1)/2$.

Once this is done rest is two drags away. Drag down D, E, F, G, H down to fill row 3. Then Shift+left click 8 times to select all of 3rd row & drag down 15–20 more iterations.

We stop according to given error margin. Suppose error margin is 10^{-4} . We will see our 14th error is 1.8310547×10^{-4} and 15th is 9.15527×10^{-5} . So, 15th $x(n+1)$ is our root.

Drawback: Other than the rate of convergence the only other drawback is discontinuities and hence we started with continuous f to begin with, i.e., for discontinuous functions naturally we can have change in sign but no roots. In our diagram, bisection method will give us α as a root too. Once we get α , we have to check if α is a point of continuity to conclude anything.

1.3.2 Regula-Falsi method

We will start with a function f and find two initial guesses x_1 & x_2 such that

$$f(x_1)f(x_2) < 0$$

We will join $(x_1, f(x_1))$ to $(x_2, f(x_2))$ via straight line. Then we find where the straight line cuts the x -axis. We come to the formulae later—first let's write down the scheme for Regula-Falsi.

We see

$$f(x_0)f(x_1) \begin{cases} > 0 & \text{use } x_0 \text{ \& } x_2 \\ < 0 & \text{use } x_0 \text{ \& } x_1 \end{cases}$$

Again draw the line joining $(x_0, f(x_0))$ & $(x_2, f(x_2))$ or $(x_1, f(x_1))$ & $(x_0, f(x_0))$.

Formulae for the algorithm: We did not assume f' exists.

$$\frac{y - y_1}{x - x_1} = \frac{y_2 - y_1}{x_2 - x_1} \implies \frac{y - f(x_1)}{x - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

$$\text{Algorithm} := \frac{y - f(x_l)}{x - x_l} = \frac{f(x_r) - f(x_l)}{x_r - x_l} \stackrel{\text{def}}{=} \frac{y - f(x_r)}{x - x_r} = \frac{f(x_r) - f(x_l)}{x_r - x_l}$$

where x_l & x_r are standard place holders for left & right end points in bracketing methods, and where x_l & x_r are either updated or maintained based on sign of $f(x_0^{(i)})f(x_l)$, & lastly

where $x_0^{(i)} := i^{\text{th}}$ guess for root is achieved by solving the above equation for x using $y = 0$, i.e.,

$$\begin{aligned} -\frac{f(x_l)}{x - x_l} &= \frac{f(x_r) - f(x_l)}{x_r - x_l} \\ \implies -\frac{f(x_l)}{f(x_r) - f(x_l)} \cdot (x_r - x_l) &= x - x_l \\ \implies x_0^{(i)} = x_l - \left(\frac{x_l - x_r}{f(x_l) - f(x_r)} \right) f(x_l) &\text{ OR } x_0^{(i)} = x_r - \left(\frac{x_l - x_r}{f(x_l) - f(x_r)} \right) f(x_r) \end{aligned}$$

Remark 1.5. We will see Newton's method & secant method that will have very similar form BUT there we will be starting with a single initial guess. Does that matter?

Yes! Regula-Falsi guarantees that we will converge to a root being one of the bracketed methods. That is not the case for Newton / Secant methods. However, we will see Newton's & Secant's methods because they are much faster than Bisection & Regula-Falsi. We write

$$|\mathcal{E}_{k+1}| = C|\mathcal{E}_k|^p \quad \text{where } \mathcal{E}_i := i^{\text{th}} \text{ error}$$

to denote a process has p -order rate of convergence. For Bisection, $C = \frac{1}{2}$ & $p = 1$ as seen already. Let's do it for Regula-Falsi.

When we get close to true root α , one of the end points gets fixed & the other end closes in on the true root. Let x_l be fixed after a few iterations and $x_r \rightarrow \alpha$. Then, we want to study how $\{x_n, x_{n+1}, \dots\}$ moves towards α . Let

$$x_l \equiv a$$

$$x_k = \alpha + \mathcal{E}_k$$

Since right end point is moving we use the second form of Regula-Falsi involving x_r for obvious reason (now x_{n+1} & x_n will both appear here in the equation):

$$\begin{aligned} x_{n+1} &= x_n - \frac{a - x_n}{f(a) - f(x_n)} f(x_n) \\ \implies \mathcal{E}_{n+1} &= \mathcal{E}_n - \frac{a - \alpha - \mathcal{E}_n}{f(a) - f(x_n)} f(x_n) \end{aligned}$$

Taylor Expansion around α gives us $f(x_n)$ that we can put in above equation:

$$\begin{aligned} f(x_n) &= f(\alpha + \mathcal{E}_n) = f(\alpha) + \mathcal{E}_n f'(\alpha) + \frac{1}{2} \mathcal{E}_n^2 f''(\alpha) + \dots \\ &\sim 0 + \mathcal{E}_n f'(\alpha) + \mathcal{O}(\mathcal{E}_n^2) \end{aligned}$$

$$\begin{aligned} \implies \lim_n \frac{\mathcal{E}_{n+1}}{\mathcal{E}_n} &= 1 - \lim_n \frac{(a - \alpha - \mathcal{E}_n)(\mathcal{E}_n f'(\alpha) + \mathcal{O}(\mathcal{E}_n^2))}{\mathcal{E}_n \{f(a) - \mathcal{E}_n f'(\alpha) - \mathcal{O}(\mathcal{E}_n^2)\}} \\ &= 1 - \lim_n \frac{(a - \alpha - \mathcal{E}_n)(f'(\alpha) + \mathcal{O}(\mathcal{E}_n^2)/\mathcal{E}_n)}{f(a) - \mathcal{E}_n f'(\alpha) - \mathcal{O}(\mathcal{E}_n^2)} \\ &= 1 - \frac{(a - \alpha)}{f(a)} f'(\alpha) := C. \end{aligned}$$

$$\mathcal{E}_{n+1} \sim C|\mathcal{E}_n|$$

In order to get a constant C we did not have to use $|\mathcal{E}_n|^2$ —which we will need for Newton's method making it faster. Moreover, C here is dependent on a ; so we might have some f such that $1 - (a - \alpha) \frac{f'(\alpha)}{f(a)}$ is less than $\frac{1}{2}$ or greater than $\frac{1}{2}$. Hence, Regula-Falsi even though

has same order as Bisection ($p = 1$) can be faster or slower than bisection. In practice, Regula-Falsi is faster since in practice we don't reach the zero, we go near it as much as needed for the sum & a process using secant lines will do that better.

Few important notes:

1. Rate of convergence analysis will always have similar structure as the one we just did. We will use the algorithm to get an equation in x_{n+1} & x_n , then put $\mathcal{E}_{n+1} + \alpha$ & $\mathcal{E}_n + \alpha$ to get an eq. in $\mathcal{E}_{n+1}$ & $\mathcal{E}_n$. Finally, do Taylor expansion of f at some point as needed by the analysis.
2. Even though the processes don't need f'' to exist, in rate analysis we will assume f'' exists. We use f', f'' in Taylor expansion and this is fine; all we need is to get an idea of how good the algorithm is compared to others. It doesn't affect us to do that for this class.
3. Improvement of Regula-Falsi—Illinois method.

We had

$$\frac{x_r f(x_l) - x_l f(x_r)}{f(x_l) - f(x_r)}$$

for Regula-Falsi; now we are going to change weightage. If left end point is unchanged we will use $\frac{1}{2}f(x_l)$ for next iteration instead of $f(x_l)$.

Similarly, if x_r is fixed, use $\frac{1}{2}f(x_r)$. Then algorithm is

$$\frac{\frac{1}{2}x_r f(x_l) - x_l f(x_r)}{\frac{1}{2}f(x_l) - f(x_r)} \quad \underline{\text{OR}} \quad \frac{x_r f(x_l) - \frac{1}{2}x_l f(x_r)}{f(x_l) - \frac{1}{2}f(x_r)}$$

As is natural from the diagram Illinois algorithm increases the odds of the next root ($n + 1^{\text{th}}$ root) appearing on the other side than where x_n did. Of course, we may have x_n, x_{n+1} both appearing on the same side (easy to draw such graphs) but nevertheless, Illinois changes the odds.

Why? Because problem with Regula-Falsi was $(a - \alpha)$ appearing in C —the one side that was stuck forever was the issue. We only needed a way to get roots on both sides of α , then we close in on α from both sides. We will skip the calculation but this works! Order of convergence of Illinois is $1.442 > 1!$ (Still not quadratic though).

4. Notice that $\frac{f(x_l) - f(x_r)}{x_l - x_r}$ is finite difference slope (discrete version of f'). We use it a lot.

1.3.3 Newton's method

This is the fastest method and it requires f to be differentiable.

This is again an iterative method. Suppose we have x_n at hand & we want to find x_{n+1} . Draw a tangent at $f(x_n)$. The line meets x -axis at x_{n+1} . Let's find the formulae for the algorithm.

Eq. of the line:

$$\begin{aligned} \frac{y_1 - y_0}{x_1 - x_0} &= m := \text{slope of tangent line} \\ \frac{f(x_n) - 0}{x_n - x_{n+1}} &= f'(x_n) \\ \implies \frac{f(x_n)}{f'(x_n)} &= x_n - x_{n+1} \implies x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \end{aligned}$$

Repeat the process with x_{n+1} to get x_{n+2} as in the diagram and we see $\{x_n\}$ sequence is getting closer to the root.

Example 1.6. We will find $\sqrt{2}$. Let's start with $f(x) = x^2 - 2$.

$$f'(x) = 2x$$

Then,

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{x_n^2 + 2}{2x_n}$$

Let's start with $x_0 = 1$.

$$x_1 = \frac{1+2}{2} = \frac{3}{2} = 1.5$$
$$x_2 = \frac{\frac{9}{4}+2}{3} = \frac{17}{12} \approx 1.416$$

We put this in computer to do the calculation for various x candidates. Naturally, at any finite step we will be at a rational number & hence none will be truly $\sqrt{2}$ but the limit will be.

Example 1.7. We have seen all polynomial examples. Let's see $f(x) = a - \frac{1}{x}$ and use Newton.

$$f'(x) = \frac{1}{x^2}$$

Then,

$$\begin{aligned} x_{n+1} &= x_n - \frac{a - \frac{1}{x_n}}{\frac{1}{x_n^2}} \\ &= x_n - x_n^2 \left(a - \frac{1}{x_n} \right) \\ &= x_n - x_n^2 a + x_n \\ &= x_n(2 - ax_n) \end{aligned}$$

So,

$$x_{n+1} = x_n(2 - ax_n).$$

We start with some value of x_0 to get the true root here that is trivially $\frac{1}{a}$. Let $a = 2$.

Let $x_0 = 1$:

$$x_1 = 1(2 - 2 \cdot 1) = 0$$
$$x_2 = 0$$

Let $x_0 = \frac{3}{4}$:

$$\begin{aligned} x_1 &= \frac{3}{4} \left(2 - 2 \cdot \frac{3}{4} \right) = \frac{3}{4} \cdot \frac{1}{2} = \frac{3}{8} = 0.375 \\ x_2 &= \frac{3}{8} \left(2 - 2 \cdot \frac{3}{8} \right) = \frac{3}{8} \cdot \frac{5}{4} = \frac{15}{32} = 0.46875 \\ x_3 &= \frac{15}{32} \left(2 - 2 \cdot \frac{15}{32} \right) = \frac{15}{32} \cdot \frac{17}{16} = \frac{255}{512} \approx 0.498046875 \\ x_4 &= \frac{255}{512} \left(2 - 2 \cdot \frac{255}{512} \right) = \frac{255}{512} \cdot \frac{257}{256} \approx 0.49999237060546875 \end{aligned}$$

!!!! Problem (What the problem is will be seen later in rate of conv. discussion).

DATA FITTING TECHNIQUES

2

2.1 Section 1

2.1.1 Subsection 1

NUMERICAL CALCULUS TECHNIQUES

3

3.1 Section 1

3.1.1 Subsection 1

NUMERICAL DIFFERENTIAL EQUATIONS

4